

Scaling Up an EKS Node Group with Terraform

Two paths: a one-time manual bump, or wiring up automatic scaling

Manually scale a managed node group

If you're using the raw `aws_eks_node_group` resource:

```
resource "aws_eks_node_group" "main" {
  cluster_name      = aws_eks_cluster.this.name
  node_group_name   = "main-node-group"
  node_role_arn     = aws_iam_role.node.arn
  subnet_ids       = var.subnet_ids

  scaling_config {
    desired_size = 6 # was 3 — bump this
    max_size     = 10
    min_size     = 2
  }

  update_config {
    max_unavailable = 1
  }
}
```

Change `desired_size` (and `max_size` if the new desired count exceeds the current ceiling), then:

```
terraform plan
terraform apply
```

If you're using the popular `terraform-aws-modules/eks/aws` module instead, the same thing looks like:

```
module "eks" {
  source = "terraform-aws-modules/eks/aws"
  version = "~> 20.0"

  cluster_name = "test-cluster"
  # ...

  eks_managed_node_groups = {
    main = {
      instance_types = ["t3.medium"]
      min_size       = 2
      max_size       = 10
      desired_size   = 6 # was 3
    }
  }
}
```

A drift gotcha if you're also running Cluster Autoscaler or Karpenter

If something else (Cluster Autoscaler, Karpenter, or manual `kubectl`/console scaling) is already adjusting the ASG's desired count at runtime, a plain `terraform apply` will fight it — Terraform will try to reset `desired_size` back to whatever's in the `.tf` file on every apply, since it doesn't know the autoscaler changed it out-of-band. The standard fix is to tell Terraform to stop tracking that one field:

```
resource "aws_eks_node_group" "main" {
  # ...
  scaling_config {
    desired_size = 3
    max_size     = 10
    min_size     = 2
  }

  lifecycle {
    ignore_changes = [scaling_config[0].desired_size]
  }
}
```

With this in place, Terraform still manages `min_size` / `max_size` (the actual scaling *bounds*), but leaves `desired_size` alone once created, since the autoscaler is the real source of truth for that value going forward. This is the right setup if you want Terraform to define capacity boundaries while something else handles moment-to-moment scaling.

If "scale up" means setting up autoscaling itself, not just bumping a number

That's a bigger piece of infrastructure — either the Kubernetes **Cluster Autoscaler** (watches for unschedulable pods and adjusts the ASG) or **Karpenter** (provisions nodes directly, no ASG required, generally the more modern recommendation on EKS). Both need IAM roles/policies via Terraform plus a Helm release or manifest applied to the cluster. Which one you want determines what the Terraform looks like:

- **Cluster Autoscaler** pairs with your existing managed node groups — it's the more "classic" approach and keeps the ASG model you already have.
- **Karpenter** replaces node-group-based scaling with just-in-time node provisioning driven by NodePool custom resources — generally faster to scale and more flexible on instance-type selection, but a different mental model than ASGs.

Follow up with which one fits your setup and the full Terraform (IAM roles, policies, Helm release) can be laid out for that path specifically.